



JSP II

1. Using scriptlet, expression and declaration
2. JSP Directives
3. Importance of scriptless JSP elements

1. Using scriptlet, expression and declaration

- These JSP elements insert java code directly in JSP
- JSP can make use of scriptlets for java code and expression for generating HTML.
- Major disadvantage is scriptlet, expression and declaration can not be embedded into one another.
- So code becomes clumpy and readability is affected.
- Example:

```
if(hh < 12)
{ %>
    <%= "<h1> GOOD MORNING </h1>" %>
<% }
else if(hh >= 12 && hh <= 16)
{ %>
    <%= "<h1> GOOD AFTERNOON </h1>" %>
<% }
```

- JSP directives are instructions provided to the JSP engine about various aspects of the page.
- They play a crucial role in configuring and managing the behavior of the JSP pages.
- There are three types of directives: page directive, include directive and taglib directive
- page directive
 - It is one of the most important and frequently needed JSP directives
 - Different attributes used in page directive are: import, language, errorPage, isErrorPage, session, contentType etc
 - Many attributes are already set with default value e.g. language="java"

- include directive
 - It allows to incorporate content from another file into a JSP page during translation phase
 - The included content becomes part of the current JSP page as if it were written directly within it.
- taglib directive
 - It allows to incorporate custom tags from external tag libraries into the JSP pages.
 - Important attributes are
 1. uri: Specifies the unique URI (usually a URL) that identifies the tag library.
 2. prefix: Assigns a short prefix to reference the tags from the library within the page.

- Scriptless JSP elements do not directly embed Java code within the page.
- Scriptless JSP elements is generally considered a best practice
- It promotes separation of concerns and better collaboration between designers and developers.
- Following are the scriptless JSP elements :
 1. Std(JSP) actions
 2. Expression Language(EL)
 3. JSTL(JSP Standard Tag Library)
 4. Custom tags

4. Steps for implementation

1. Install latest redux library
2. Create store by using `configureStore()` function
3. Wrap `<App />` component in `<Provider />` element
4. Create slice by using `createSlice()` function needs object as argument which has keys - `name`, `initialState(Object)`, `reducers(object)`.
5. Once the slice is ready, make the entry in store
6. Access state in the component by using `useSelector()` hook.
7. Use `useDispatch` for modifying the state